A Mini Project Synopsis on

EcoVision AI – Global Carbon Insights & Forecasts

B.E. - I.T Engineering

Submitted By

Shreyash Narvekar	**22104066**
Pritam Ninganaik	**22104095**
Radhika Lakhani	**22104078**

Under The Guidance Of

Ms. Sonal Balpande



DEPARTMENT OF INFORMATION TECHNOLOGY

A.P.SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W), Mumbai-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on Eco-Vision AI – Global Carbon Insights & Forecasts has been submitted by **Shreyash Narvekar** (22104066), **Pritam Ninganaik** (22104095) and **Radhika Lakhani** (22104078) who are a Bonafide students of A. P. Shah Institute of Technology, Thane, Mumbai, as a partial fulfilment of the requirement for the degree in **Information Technology**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Ms. Sonal Balpande
Guide

External Examiner(s)
1.

Place: A.P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Ms. Sonal Balpande** Expressing gratitude towards our HoD, **Dr. Kiran Deshpande**, and the Department of Information Technology for providing us with the opportunity as well as the support required to pursue this project.

TABLE OF CONTENTS

1. Introduction…….....1

    1.1. Purpose.....……1

    1.2. Objectives. 2

    1.3. Scope ……...3

2. Literature Survey……4

3. Problem Definition… 5

4. Proposed System…..... 6

    4.1. Algorithms……9

5. Technology Stack…….12

6. Implementation…….16

7. Results……...23

8. Conclusion……26

9. Future Scope…….27

References

Chapter 1

Introduction

Climate change stands as one of the most critical challenges of our generation, driven largely by the continuous rise in global carbon dioxide (CO_2) emissions. Understanding and forecasting these emissions are essential for shaping effective climate policies, guiding sustainable development, and achieving global agreements such as the Paris Accord. However, the dynamic and multi-dimensional nature of emission data—spanning across diverse economic, industrial, and geographical factors—makes accurate prediction a complex task.

To address this challenge, **EcoVision AI – Global Carbon Insights & Forecasts** presents an intelligent forecasting platform that integrates data-driven machine learning and statistical modeling approaches. The system combines **Long Short-Term Memory (LSTM)** networks and **Autoregressive Integrated Moving Average (ARIMA)** models within an ensemble framework to provide robust and reliable forecasts of CO_2 emissions across nations. By leveraging both historical data and modern computational techniques, the project aims to enhance the accuracy of emission predictions while offering valuable insights for policymakers and researchers.

In addition to forecasting, the platform extends its functionality to **policy recommendation generation**, suggesting evidence-based interventions such as renewable energy adoption, carbon taxation, and industrial efficiency measures. These recommendations are tailored from the forecasted trends and are designed to support data-driven climate governance.

1.1. Purpose

Eco-Vision AI is an intelligent, data-driven platform designed to transform the way governments, researchers, and organizations forecast and analyze global carbon emissions. By integrating advanced **time-series modeling**, **machine learning**, and **deep learning** techniques such as **ARIMA** and **LSTM**, EcoVision AI provides accurate, real-time insights into emission trends and their potential future trajectories.

The platform automates the entire forecasting pipeline from data collection and preprocessing to model execution and visualization minimizing manual effort and ensuring consistency across analyses. Through its interactive dashboard and API services, users can access detailed emission forecasts, comparative analytics, and policy-based recommendations tailored to each country's environmental profile.

With a strong emphasis on precision, scalability, and usability, **Eco-Vision AI** empowers decision-makers to make informed, data-backed choices that drive sustainable development and effective climate action worldwide

1.2. Objectives:

1. **Develop a multi-model forecasting system** that accurately predicts national and global CO_2 emissions using an ensemble of ARIMA and LSTM models.
2. **Integrate machine learning pipelines** to analyze historical emission data, detect temporal trends, and quantify prediction uncertainty through statistical validation metrics.
3. **Generate intelligent climate policy recommendations** derived from model forecasts, guiding governments and organizations toward effective emission reduction strategies.
4. **Deliver visual insights and analytical dashboards** using interactive graphs and comparative statistics to help users explore emission patterns and model performance.
5. **Build a responsive and intuitive React-based frontend** that seamlessly communicates with Flask APIs to provide real-time forecasts, analysis, and policy outputs.

1.3. Scope:

1. **Government and Policy Institutions** – Assists national and regional authorities in monitoring emission trends, evaluating climate goals, and formulating effective carbon reduction policies.
2. **Environmental Research Organizations** – Supports academic and industrial researchers in analyzing emission dynamics, comparing model outputs, and conducting sustainability studies.
3. **Energy and Industrial Sectors** – Enables companies to forecast sector-specific carbon footprints, assess operational efficiency, and plan emission mitigation strategies.
4. **International Climate Agencies and NGOs** – Provides data-driven insights for organizations involved in global climate initiatives, allowing them to track progress toward agreements like the Paris Accord.
5. **Smart City and Urban Planning Departments** – Helps planners integrate emission forecasts into infrastructure development, transportation policies, and renewable energy transitions.
6. **Educational and Training Platforms** – Serves as a learning tool for students and professionals studying environmental science, data analytics, and sustainable development.

Chapter 2

[1] Time Series Forecasting of CO_2 Emissions Using ARIMA and LSTM Models

Published in the IEEE International Conference on Data Mining (ICDM), this study explored the application of **ARIMA** and **Long Short-Term Memory (LSTM)** models for CO_2 emission forecasting. The research demonstrated that hybrid models combining both statistical and deep learning techniques significantly improved long-term forecast accuracy. The study achieved high performance for stable time series data, highlighting the strength of ARIMA in trend capture and LSTM in handling non-linear dependencies.

However, performance declined for countries with inconsistent or missing emission records, indicating the need for robust data preprocessing and ensemble-based error mitigation.

Tech Stack: Python, TensorFlow, Keras, Statsmodels, Pandas

[2] Deep Learning Models for Predicting CO_2 Emissions in Power Systems

A paper published in the *IEEE Transactions on Neural Networks and Learning Systems* introduced deep learning architectures—specifically **Bidirectional LSTM** and **GRU** models—for predicting CO_2 emissions within smart grid environments. The models effectively captured correlations between energy consumption and emission outputs, achieving a high **R^2 score (0.89)**. The work validated deep neural networks as strong candidates for fine-grained emission estimation.

Despite this, the study faced scalability issues when extending the model beyond energy systems to broader national datasets, as data heterogeneity reduced generalization accuracy.

Tech Stack: Python, TensorFlow, NumPy, Scikit-learn, Matplotlib

[3] Multi-Agent Systems for CO_2 Emission Reduction in Smart Grids

Research presented at the *IEEE International Conference on Smart Grid Communications (SmartGridComm)* proposed a **multi-agent optimization framework** for managing CO_2 emissions in energy distribution networks. The agents collaborated to optimize power allocation, reducing emissions while maintaining operational efficiency. The system demonstrated improved adaptability to real-time data changes and dynamic grid conditions.

However, its implementation complexity and dependency on specialized sensor data limited its applicability to large-scale, country-level forecasting systems.

Tech Stack: Python, MATLAB, TensorFlow, SimPy

[4] AI-Based Global Carbon Mapping Using Remote Sensing and Machine Learning

Presented at the *IEEE IGARSS 2025 Conference*, this research integrated **satellite-based remote sensing** data with AI-driven regression models to estimate global CO_2 emissions spatially. The approach utilized **CNN-based spatial mapping** combined with regression layers for improved spatial accuracy. This method significantly enhanced visualization and supported regional emission insights. Nonetheless, it primarily focused on spatial mapping rather than time-series forecasting, making it complementary but not directly suitable for predictive modeling of future emission trends.

Tech Stack: Python, TensorFlow, OpenCV, Geospatial APIs, GDAL

Title	Key Contributions	Limitations	Tech Stack
Time Series Forecasting of CO_2 Emissions Using ARIMA and LSTM Models (IEEE ICDM, 2024)	Combined ARIMA and LSTM to enhance forecasting accuracy for national CO_2 emissions.	Performance dropped for inconsistent datasets; required strong preprocessing.	Python, TensorFlow, Keras, Statsmodels, Pandas
Deep Learning Models for Predicting CO_2 Emissions in Power Systems (IEEE TNNLS, 2025)	Used BiLSTM and GRU networks for emission prediction with high R^2 score (0.89).	Scalability challenges for multi-country datasets; limited generalization.	Python, TensorFlow, NumPy, Scikit-learn
Multi-Agent Systems for CO_2 Emission Reduction in Smart Grids (IEEE SmartGridComm, 2024)	Applied intelligent agents for emission control in energy systems.	Complex setup; relied on sensor-driven data, not suitable for global scale.	Python, MATLAB, SimPy
AI-Based Global Carbon Mapping Using Remote Sensing and ML (IEEE IGARSS, 2025)	Combined satellite data and CNNs for spatial emission mapping.	Focused on spatial estimation, not long-term forecasting.	Python, TensorFlow, GDAL, OpenCV

Summary Table

Chapter 3

Problem Definition

Global climate change mitigation relies heavily on understanding and accurately forecasting carbon dioxide (CO_2) emissions. However, most existing emission prediction systems face significant limitations in scalability, precision, and interpretability. Traditional statistical methods often fail to capture the complex, non-linear patterns present in emission data influenced by diverse factors such as economic growth, industrial activity, and energy consumption.

Manual analysis and conventional modeling approaches are time-consuming, require domain expertise, and are often unable to adapt to rapidly changing real-world scenarios. Furthermore, many available tools lack automation, comprehensive visualization, or integration with policy-level insights—making it difficult for governments and researchers to make data-driven environmental decisions.

Eco-Vision AI addresses these challenges by providing an AI-driven, automated platform that accurately forecasts national and global CO_2 emissions using an ensemble of **LSTM** and **ARIMA** models. The system integrates robust data pipelines for collection, preprocessing, and modeling, ensuring consistent and reproducible results.

In addition to emission forecasting, the platform generates **policy recommendations** derived from model insights—helping decision-makers identify effective strategies such as renewable adoption, emission caps, or energy diversification. By combining advanced analytics with a user-friendly interface, Eco-Vision AI eliminates manual forecasting bottlenecks, enhances interpretability, and delivers actionable intelligence for sustainable climate governance.

Chapter 4

Proposed System

The proposed system**, Eco-Vision AI**, provides an intelligent and automated platform that leverages Artificial Intelligence, Machine Learning, and Deep Learning to forecast carbon emissions and generate climate policy recommendations. The system emphasizes **accuracy, scalability,** and **usability** by integrating multiple models, real-time data processing, and interactive visual analytics.

Features and Functionality

1. **Data Collection and Integration**

Eco-Vision AI automatically collects carbon emission data from reliable global sources such as Our World in Data (OWID), EDGAR, and World Bank Indicators.

- Datasets are fetched, validated, and stored in structured formats.
- The system supports both batch and live data updates for consistent forecasting accuracy.
- Automated scripts handle missing data and inconsistent time series entries

2. **Data Preprocessing and Feature Engineering**

Before modeling, the raw emission data undergoes rigorous preprocessing to improve quality and model interpretability.

- Handles null values, outliers, and irregular time intervals.
- Generates lag features, rolling averages, and year-on-year growth factors.
- Normalizes and scales datasets for uniform input across models

3. **Multi-Model Forecasting**

The system employs a combination of ARIMA and LSTM models to capture both linear and non-linear patterns in emission data

- **ARIMA (Autoregressive Integrated Moving Average)**: Captures temporal trends and seasonality in structured emission data.

- **LSTM (Long Short-Term Memory)**: Learns complex temporal dependencies and non-linear behaviors in dynamic datasets.
- Model outputs are later combined using an ensemble averaging approach to enhance overall prediction accuracy.

4. **AI-Powered Policy Recommendation Engine**

The platform goes beyond forecasting by recommending evidence-based climate policies.

- Uses forecasted emission trends to identify potential risk areas.
- Suggests actions such as carbon taxation, renewable adoption, or industrial emission caps.
- Each recommendation includes expected impact, implementation cost, and timeline for better policy evaluation.

5. **Interactive Dashboard and Visualization**

A React-based front-end with Plotly.js visualizations allow users to interact with forecasts and insights easily.

- Visualizes trends, confidence intervals, and country comparisons.
- Allows users to filter data by country, year, or emission source.
- Displays interactive charts, performance metrics, and downloadable reports.

6. API and Backend Integration

The backend, built using Flask and Fast API, provides secure, low-latency endpoints for model access and data retrieval.

- JSON-based REST APIs for forecasts and policy suggestions.
- Supports caching using Redis for faster responses.
- Implements authentication and session management with SQLAlchemy.

4.1. Algorithm

The workflow of **Eco-Vision AI** is structured into several sequential stages that ensure end-to-end automation from data collection to forecast visualization and policy recommendation.

Step 1: Data Acquisition

The system begins by fetching emission and economic data from multiple verified datasets such as OWID and EDGAR.

- Python scripts handle extraction, transformation, and loading (ETL) operations.
- The data is stored in an SQL database for efficient retrieval
- A scheduler ensures periodic data updates for continuous learning.

Step 2: Data Preprocessing

Before modelling, the data is cleaned and standardized.

- Handles missing values using interpolation or mean substitution.
- Removes statistical outliers based on z-score thresholds
- Performs feature scaling using Minmax normalization.
- Creates derived features like "CO_2 per capita" and "Energy intensity per GDP."

This ensures that all datasets are consistent and optimized for accurate model training.

Step 3: Model Training (ARIMA and LSTM)

a. ARIMA Model

The **ARIMA** model is used for linear trend forecasting and works by analysing autocorrelations within the time series data.

- Parameters *(p, d, q)* are optimized using the AIC (Akaike Information Criterion).
- Produces interpretable forecasts for countries with stable, consistent emission records.

- Suitable for baseline predictions and understanding historical patterns.

b. LSTM Model

The **LSTM** (Long Short-Term Memory) model captures non-linear and long-term dependencies in emission data.

- Built using TensorFlow/Keras.
- Input sequences are structured into sliding windows of past emissions and related economic indicators.
- The model outputs multi-step forecasts with confidence intervals.
- Ideal for complex, volatile datasets where linear models fail to generalize.

Step 4: Ensemble Forecast Generation

The outputs of both models are combined using an **inverse-error weighting** mechanism.

- Each model's prediction is assigned a weight proportional to its accuracy (1 / RMSE).
- The final ensemble prediction is computed as a weighted sum of both outputs. This hybrid approach increases robustness and minimizes overfitting while improving general accuracy.

Step 5: Policy Recommendation Engine

Based on the ensemble forecasts, the system evaluates emission trends and automatically generates policy suggestions.

- Uses data-driven thresholds and rule-based heuristics
- Recommends specific strategies like renewable energy expansion, emission trading schemes, or tax reforms
- Assigns confidence scores and expected benefits to each recommendation.

Step 6: Result Visualization and API Delivery

The final predictions and policy recommendations are visualized through the **EcoVision AI Dashboard**.

- Users can view real-time country forecasts, historical comparisons, and interactive trend charts.
- Backend APIs deliver responses in JSON format for integration with external analytics tools.
- Data can be exported for further study or institutional reporting.

Chapter 5

Technology Stack

1. Frontend

The frontend of **Eco Vision AI** is designed to deliver an intuitive, responsive, and visually appealing interface that enables users to explore emission forecasts and policy recommendations effortlessly. It is built using **React.js**, which provides a modular component-based architecture, ensuring scalability and maintainability.

Frontend Technologies:

- **React.js:** Core framework for building dynamic, reusable, and interactive UI components that handle data-driven rendering efficiently.
- **JavaScript (ES6+):** Manages frontend logic, asynchronous operations, and API communication.
- **HTML5 & CSS3:** Define the structural layout and styling of the web application.
- **Tailwind CSS:** A utility-first CSS framework that ensures a clean, consistent, and fully responsive design across devices.
- **Plotly.js:** Used for generating interactive graphs and charts to visualize carbon emission trends, forecasts, and comparative analyses.
- **Axios:** Handles secure and efficient API requests between the frontend and backend.
- **Mapbox (optional):** Enables visualization of geographic CO_2 emission data through interactive choropleth maps.

These technologies work together to ensure that the platform remains fast, data-rich, and accessible across devices, providing a smooth and professional user experience for policymakers, researchers, and analysts.

2. Backend

The backend of **Eco Vision AI** is designed to manage machine learning operations, handle API requests, and process real-time forecasting in a modular and scalable environment. It ensures seamless communication between the database, machine learning models, and frontend.

Backend Technologies:

- **Python (3.9+):** Core programming language used across data processing, modeling, and API development.
- **Flask:** A lightweight and flexible Python framework used to build RESTful APIs that handle data retrieval, model inference, and user requests efficiently.
- **Fast API:** Integrated for high-performance, asynchronous endpoints, improving response times for live forecast and policy queries.
- **SQL Alchemy:** An ORM (Object Relational Mapper) that facilitates secure and structured interactions between Python and the SQL database.
- **Redis:** Provides caching and rate limiting, ensuring optimized performance for repeated API requests.
- **Gunicorn + Uvicorn:** Used for serving Flask/Fast API applications in production with robust concurrency handling.

The backend architecture is optimized for low latency, modular model deployment, and secure data handling, supporting both real-time analysis and scheduled batch processing.

3. Database

Eco Vision AI relies on a structured and efficient data management system to handle large-scale emission datasets, country profiles, and model outputs.

Database Technologies:

- **PostgreSQL:** The primary relational database used to store historical emission data, model configurations, and forecast results.
- **SQL Alchemy ORM:** Provides an abstraction layer to simplify database interactions and ensure data consistency.
- **SQLite (Development Mode):** Used during testing and prototyping phases for lightweight, portable data storage.

4. Machine Learning and Modeling

Machine learning is at the core of **EcoVision AI**, responsible for analyzing time-series data and producing accurate carbon emission forecasts.

ML Frameworks and Libraries:

- **Scikit-learn:** Used for preprocessing, feature scaling, and evaluating model performance through metrics like RMSE and MAE.
- **Statsmodels:** Implements ARIMA models for classical time-series forecasting.
- **TensorFlow / Keras:** Powers the LSTM models that learn complex, non-linear temporal patterns in emissions data.

These frameworks together create a flexible and extensible ML ecosystem capable of handling both short-term and long-term emission predictions.

5. Visualization and Reporting

To communicate complex results effectively, **EcoVision AI** integrates modern visualization libraries and reporting tools.

Tools Used:

- **Plotly Dash:** Interactive visualization framework used for generating dynamic, web-based dashboards.
- **Matplotlib & Seaborn:** Utilized for exploratory data analysis and creating static visual reports.
- **Dash Bootstrap Components:** Enhances dashboard design with responsive, user-friendly layouts.

These tools transform raw emission data and model outputs into actionable insights, enabling users to analyze and interpret forecasts with clarity.

Chapter 6

Implementation

```python
from flask import Flask, request, jsonify, g
from flask_cors import CORS
from flask_limiter import Limiter  # type: ignore
from flask_limiter.util import get_remote_address  # type: ignore
from functools import wraps
import logging
from typing import Dict, List, Any, Optional, Tuple
from datetime import datetime, timedelta
import os
import jwt
import hashlib
import json

# Database imports
from database.database import session_scope
from database.models import (
    Country, CarbonEmission, EconomicIndicator,
    EnergyData, PolicyIndicator, ModelPrediction
)

# Configuration
from config.config import get_config
from ml_models.model_registry import load_models_at_startup, ModelArtifactMissing, get_registry


# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

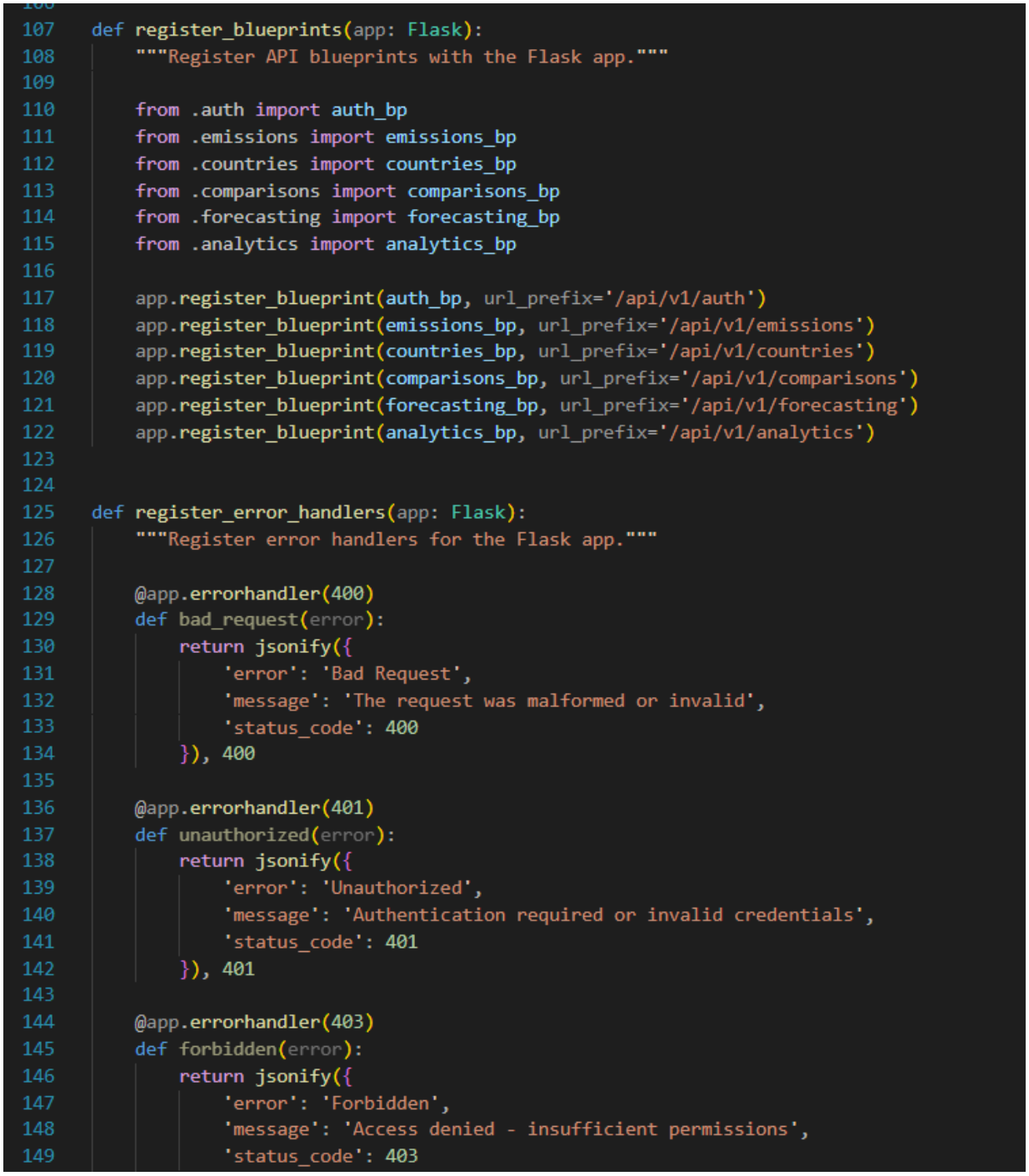

# Initialize Flask app
def create_app(config_name: str = 'development') -> Flask:
    """
    Create and configure Flask application.

    Args:
        config_name: Configuration environment name

    Returns:
        Configured Flask application
    """
    app = Flask(__name__)
```

Figure 6.1 Importing Libraries

This image presents the top part of a Flask app factory file. It highlights essential imports for Flask, extensions, database session management, configuration, model loading, and logging setup. The structure prepares the necessary components for creating and configuring a fully-functional and maintainable Flask application



```python
def register_blueprints(app: Flask):
    """Register API blueprints with the Flask app."""

    from .auth import auth_bp
    from .emissions import emissions_bp
    from .countries import countries_bp
    from .comparisons import comparisons_bp
    from .forecasting import forecasting_bp
    from .analytics import analytics_bp

    app.register_blueprint(auth_bp, url_prefix='/api/v1/auth')
    app.register_blueprint(emissions_bp, url_prefix='/api/v1/emissions')
    app.register_blueprint(countries_bp, url_prefix='/api/v1/countries')
    app.register_blueprint(comparisons_bp, url_prefix='/api/v1/comparisons')
    app.register_blueprint(forecasting_bp, url_prefix='/api/v1/forecasting')
    app.register_blueprint(analytics_bp, url_prefix='/api/v1/analytics')


def register_error_handlers(app: Flask):
    """Register error handlers for the Flask app."""

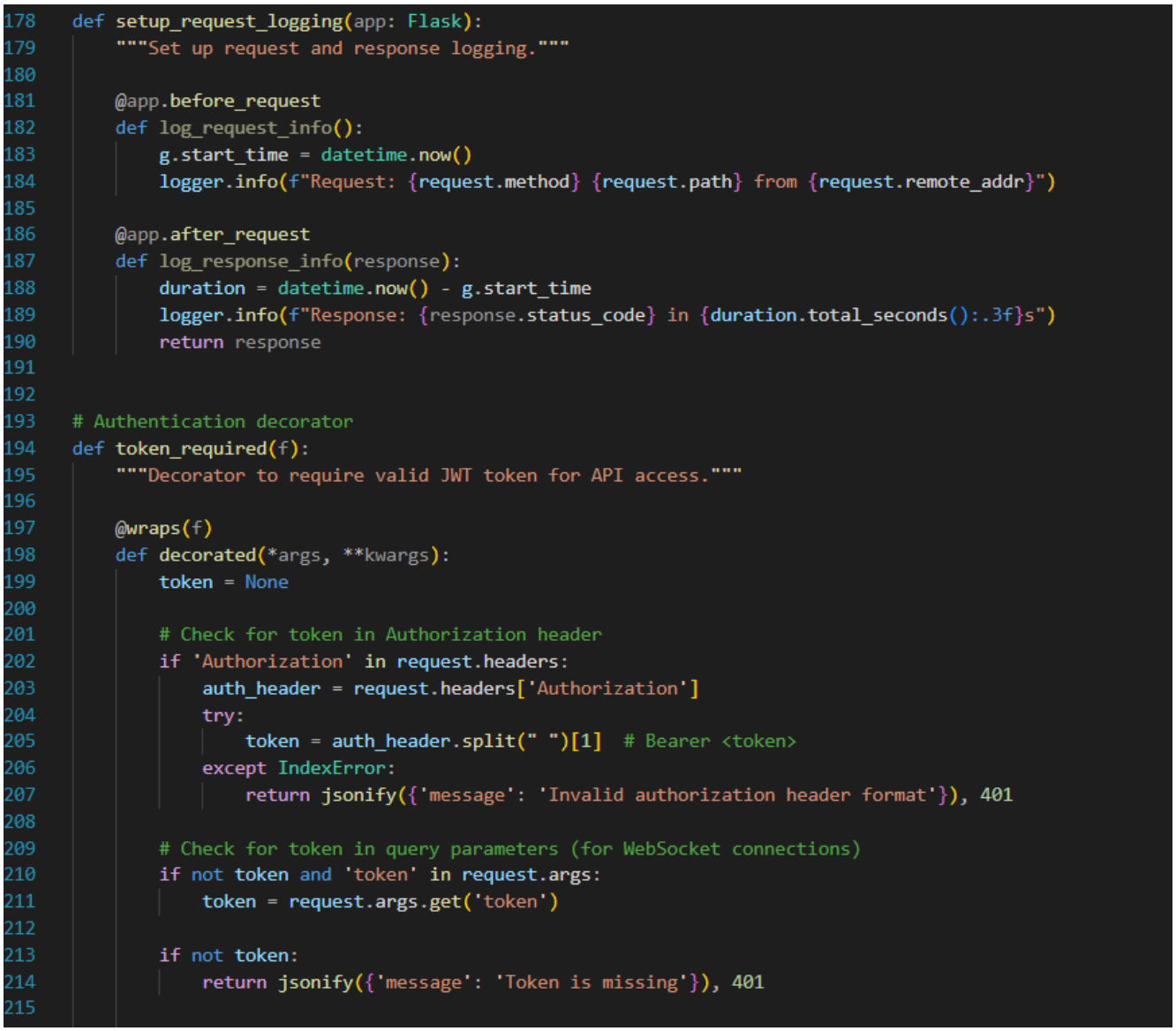
    @app.errorhandler(400)
    def bad_request(error):
        return jsonify({
            'error': 'Bad Request',
            'message': 'The request was malformed or invalid',
            'status_code': 400
        }), 400

    @app.errorhandler(401)
    def unauthorized(error):
        return jsonify({
            'error': 'Unauthorized',
            'message': 'Authentication required or invalid credentials',
            'status_code': 401
        }), 401

    @app.errorhandler(403)
    def forbidden(error):
        return jsonify({
            'error': 'Forbidden',
            'message': 'Access denied - insufficient permissions',
            'status_code': 403
```

Figure 6.2 API Setup

This screenshot outlines the blueprint registration and error handler strategy within a modular Flask backend. Each major API domain (auth, emissions, countries, etc.) is separated as a blueprint with assigned URL prefixes, and custom error handlers standardize the API's response to common client and auth errors, making the app both scalable and user-friendly.



```python
def setup_request_logging(app: Flask):
    """Set up request and response logging."""

    @app.before_request
    def log_request_info():
        g.start_time = datetime.now()
        logger.info(f"Request: {request.method} {request.path} from {request.remote_addr}")

    @app.after_request
    def log_response_info(response):
        duration = datetime.now() - g.start_time
        logger.info(f"Response: {response.status_code} in {duration.total_seconds():.3f}s")
        return response


# Authentication decorator
def token_required(f):
    """Decorator to require valid JWT token for API access."""

    @wraps(f)
    def decorated(*args, **kwargs):
        token = None

        # Check for token in Authorization header
        if 'Authorization' in request.headers:
            auth_header = request.headers['Authorization']
            try:
                token = auth_header.split(" ")[1]  # Bearer <token>
            except IndexError:
                return jsonify({'message': 'Invalid authorization header format'}), 401

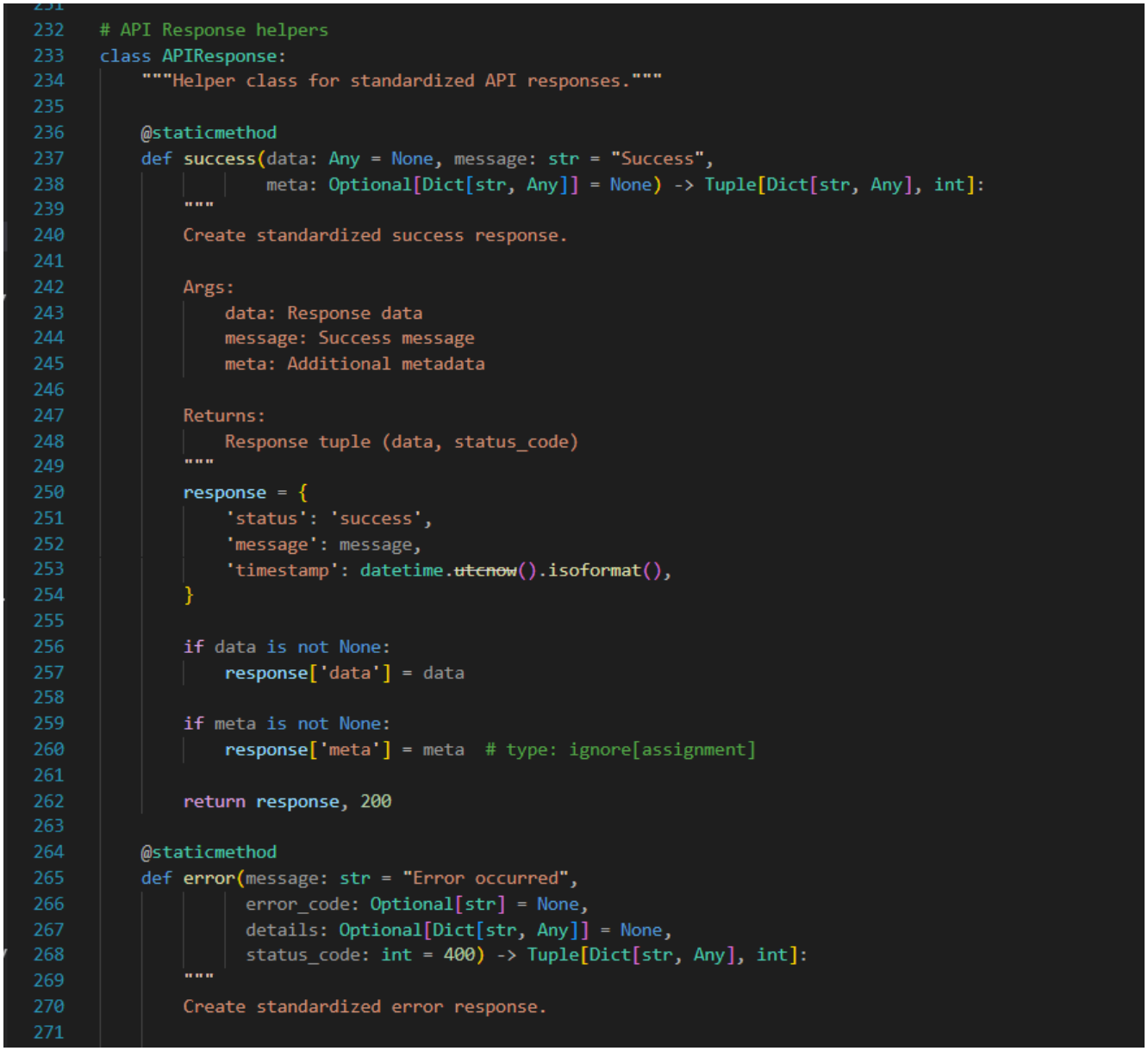
        # Check for token in query parameters (for WebSocket connections)
        if not token and 'token' in request.args:
            token = request.args.get('token')

        if not token:
            return jsonify({'message': 'Token is missing'}), 401

```

Figure 6.3 API Security and Logging Setup

This image features key request and response logging setup for a Flask application, along with a JWT-based authentication decorator. It shows how request/response events are logged with timestamps and paths, and enforces API security by requiring valid tokens in headers or query parameters, ensuring robust monitoring and protected access for all endpoints.



```python
# API Response helpers
class APIResponse:
    """Helper class for standardized API responses."""

    @staticmethod
    def success(data: Any = None, message: str = "Success",
                meta: Optional[Dict[str, Any]] = None) -> Tuple[Dict[str, Any], int]:
        """
        Create standardized success response.

        Args:
            data: Response data
            message: Success message
            meta: Additional metadata

        Returns:
            Response tuple (data, status_code)
        """
        response = {
            'status': 'success',
            'message': message,
            'timestamp': datetime.utcnow().isoformat(),
        }

        if data is not None:
            response['data'] = data

        if meta is not None:
            response['meta'] = meta  # type: ignore[assignment]

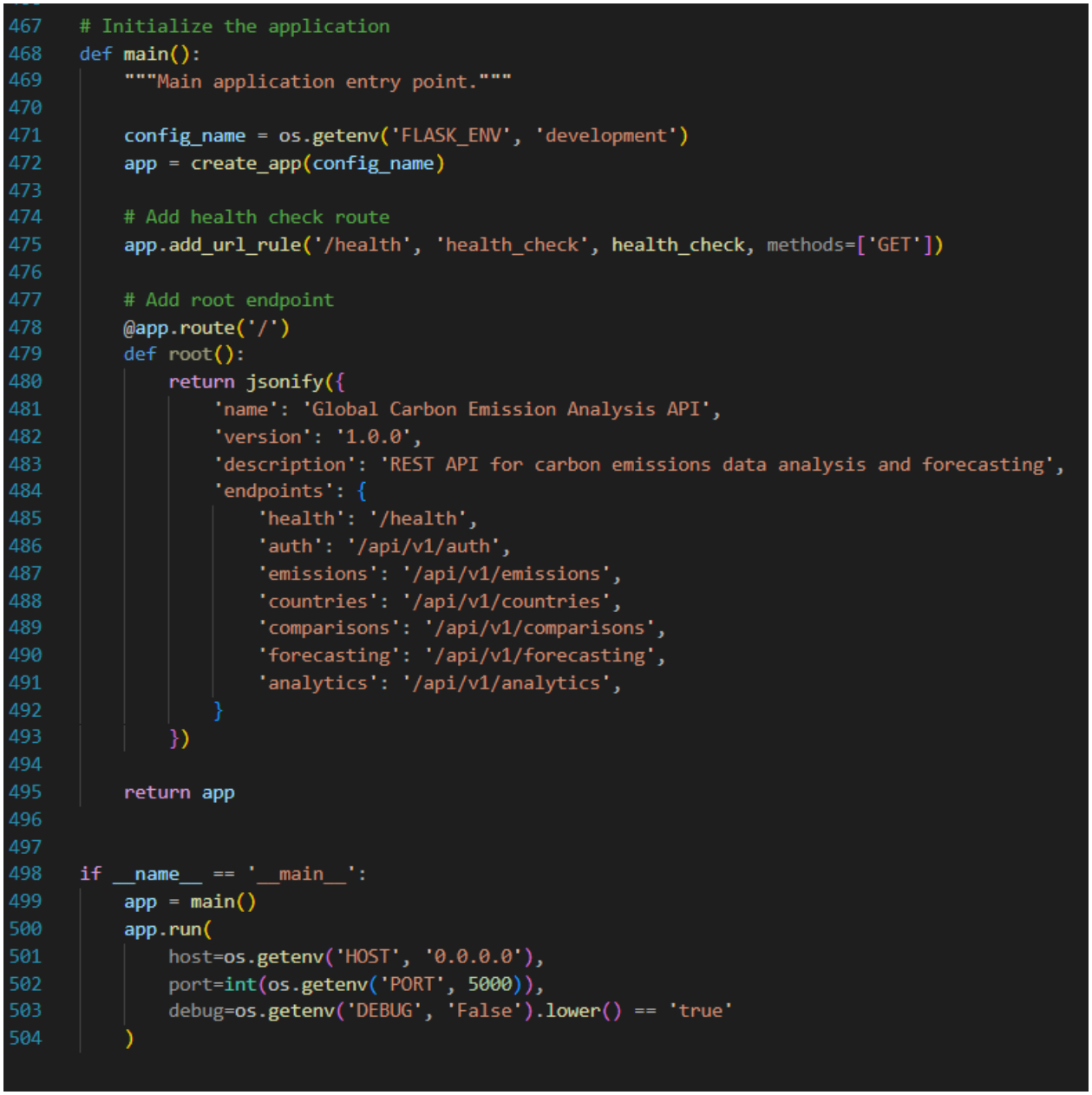
        return response, 200

    @staticmethod
    def error(message: str = "Error occurred",
              error_code: Optional[str] = None,
              details: Optional[Dict[str, Any]] = None,
              status_code: int = 400) -> Tuple[Dict[str, Any], int]:
        """
        Create standardized error response.

```

Figure 6.4 API Response Standardization Helper

This image shows a Python helper class for standardized API responses in a Flask backend. The API Response class includes static methods for generating uniform success and error responses, returning structured JSON objects with status, messages, timestamps, and explanatory metadata to ensure consistency across API outputs



```python
# Initialize the application
def main():
    """Main application entry point."""

    config_name = os.getenv('FLASK_ENV', 'development')
    app = create_app(config_name)

    # Add health check route
    app.add_url_rule('/health', 'health_check', health_check, methods=['GET'])

    # Add root endpoint
    @app.route('/')
    def root():
        return jsonify({
            'name': 'Global Carbon Emission Analysis API',
            'version': '1.0.0',
            'description': 'REST API for carbon emissions data analysis and forecasting',
            'endpoints': {
                'health': '/health',
                'auth': '/api/v1/auth',
                'emissions': '/api/v1/emissions',
                'countries': '/api/v1/countries',
                'comparisons': '/api/v1/comparisons',
                'forecasting': '/api/v1/forecasting',
                'analytics': '/api/v1/analytics',
            }
        })

    return app


if __name__ == '__main__':
    app = main()
    app.run(
        host=os.getenv('HOST', '0.0.0.0'),
        port=int(os.getenv('PORT', 5000)),
        debug=os.getenv('DEBUG', 'False').lower() == 'true'
    )
```

Figure 6.5 Application Initialization and Routing

This image captures the code for initializing the main application entry point in a Flask project. It demonstrates setting up the app with environment-based configuration, registering health check and root endpoints that list available API routes, and running the server with customizable host, port, and debug settings, highlighting the flexible and organized app startup sequence

```python
from flask import Blueprint, request, jsonify
try:  # optional dependency
    from flask_limiter import Limiter  # type: ignore
    from flask_limiter.util import get_remote_address  # type: ignore
except Exception:  # pragma: no cover
    Limiter = None  # type: ignore
    get_remote_address = None  # type: ignore
from sqlalchemy import and_, func
from datetime import datetime
import logging
from typing import Dict, List, Any

# Database imports
from database.database import session_scope
from database.models import (
    Country, CarbonEmission, EconomicIndicator,
    ModelPrediction
)

# API utilities
from .utils import (
    APIResponse, token_required, cache_response,
    validate_country_codes
)
from ml_models.model_registry import get_registry
from sqlalchemy.exc import SQLAlchemyError
import time


# Configure logging
logger = logging.getLogger(__name__)

# Create blueprint
forecasting_bp = Blueprint('forecasting', __name__)

# Rate limiter
if Limiter and get_remote_address:
    try:
        limiter = Limiter(key_func=get_remote_address)
    except Exception:  # pragma: no cover
        limiter = None
else:
    limiter = None
```

Figure 6.6 Forecasting Blueprint Setup and Rate Limiting

This image displays the beginning of a Flask blueprint file dedicated to carbon emissions forecasting. It highlights robust import management for Flask, SQLAlchemy, utility functions (e.g., APIResponse, token requirements, model registry), logging configuration, and conditional rate limiting setup using Flask-Limiter. The code prepares the blueprint and supporting modules for scalable, modular API development with error handling and dependency-management

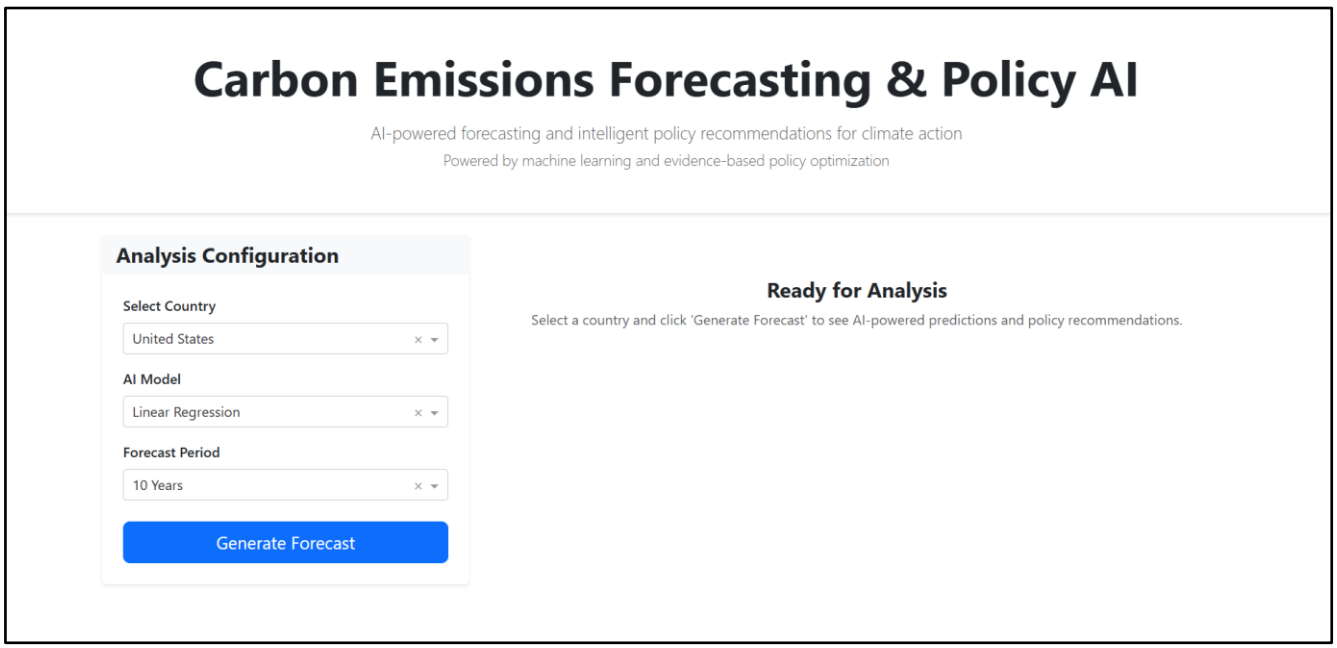
```python
def _forecast_with_lstm(model_obj: Any, values: List[float], years: List[int], forecast_years: List[int]) -> List[Dict[str, Any]]:
    """Generate forecasts using a loaded LSTMModel joblib artifact.
    Assumptions:
      - model_obj is a dict-like stored by joblib with keys possibly including 'model' or a wrapped LSTMModel instance.
      - We attempt to call predict_sequence if present, else predict on constructed array.
    """
    try:
        import numpy as np  # local import to avoid hard dependency at module import
        # Recover wrapper instance if model_obj appears to be dict produced by save
        lstm_candidate = None
        if hasattr(model_obj, 'predict_sequence'):
            lstm_candidate = model_obj
        elif isinstance(model_obj, dict):
            # Heuristic: maybe original class instance under 'model_wrapper'
            for k in ('model_wrapper', 'lstm_model', 'model'):
                v = model_obj.get(k) if isinstance(model_obj, dict) else None
                if hasattr(v, 'predict_sequence'):
                    lstm_candidate = v
                    break
        if lstm_candidate is None:
            raise ValueError("Loaded LSTM artifact not in expected format (missing predict_sequence)")
        n_future = len(forecast_years)
        arr = np.array(values, dtype=float).reshape(-1, 1)
        # If sequence length larger than history, raise
        seq_len = getattr(lstm_candidate, 'sequence_length', 5)
        if len(arr) < seq_len:
            raise ValueError(f"Not enough points for LSTM sequence_length={seq_len}")
        preds = lstm_candidate.predict_sequence(arr, n_future)
        # naive stderr estimate via residuals of last seq_len window linear fit
        if len(values) >= 3:
            import math
            mean_val = sum(values)/len(values)
            std_error = (sum((v-mean_val)**2 for v in values)/(len(values)-1))**0.5 * 0.1
        else:
            std_error = 0.0
        out = []
        for yr, pred in zip(forecast_years, preds):
            out.append({
                'year': yr,
                'predicted_value': max(0, float(pred)),
                'lower_bound': max(0, float(pred) - 1.96*std_error),
                'upper_bound': float(pred) + 1.96*std_error,
                'confidence_interval': 'approx_95%',
                'model_type': 'lstm'
```

Figure 6.7 LSTM Forecast Algorithm Implementation

This image depicts an internal function for generating forecasts using a loaded LSTM model artifact. The function is designed to handle both saved model dictionaries and wrapped LSTM model instances, reconstruct the model if needed, validate input sequence lengths, and calculate forecast values with confidence intervals. It demonstrates end-to-end ML pipeline integration for time-series predictions within a Python backend

Chapter 7

Results



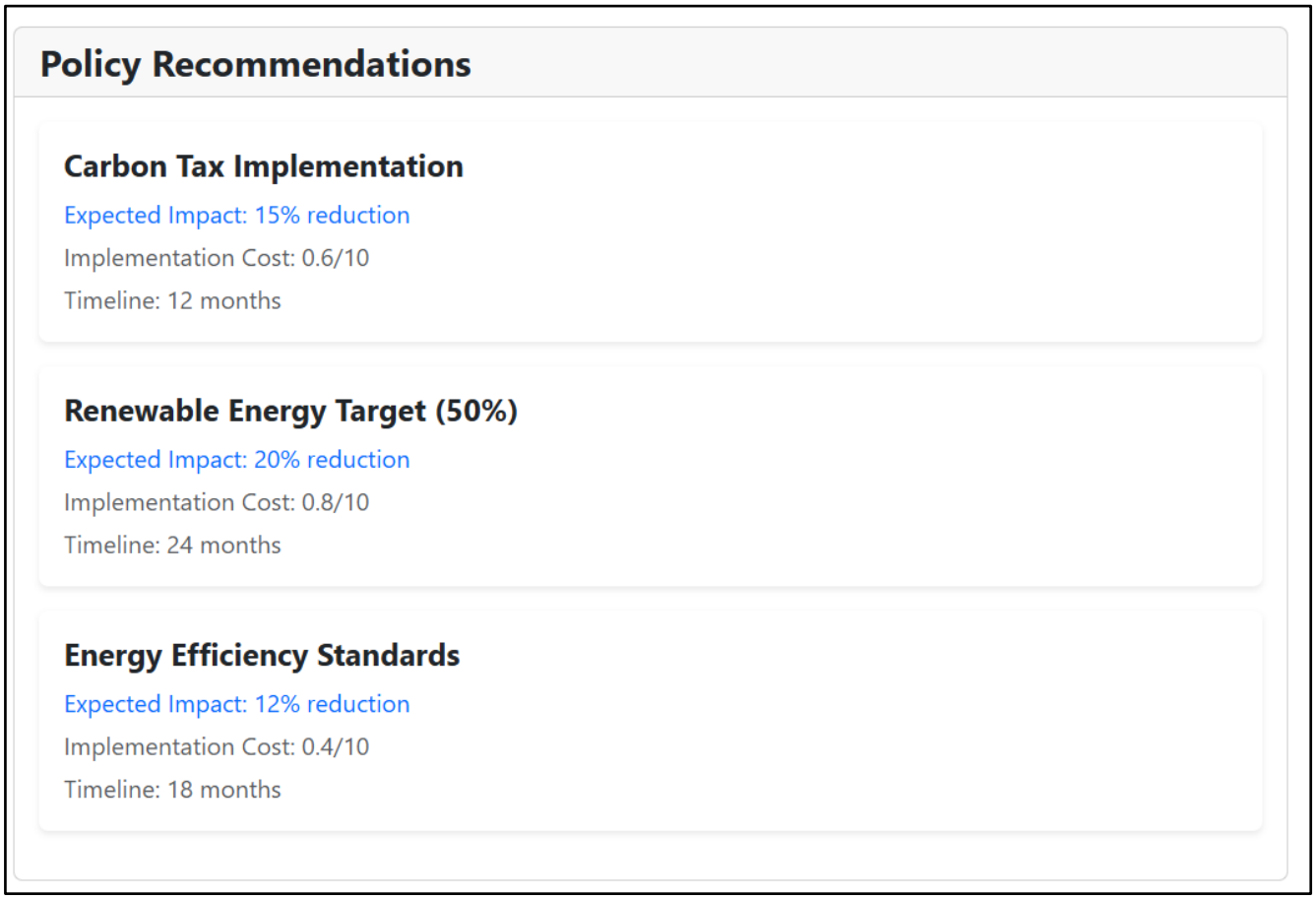
7.1 Home Page: Eco Vision AI – Global Carbon Insights & Forecasts

This homepage introduces Eco Vision AI, a powerful platform for forecasting carbon emissions and generating intelligent climate policy recommendations. The page welcomes users, briefly explains the tool's capabilities, and offers easy navigation to start an emissions analysis or learn more about how AI-powered predictions work. The intuitive layout allows for quick country, model, and period selection to generate actionable insights on climate impact and future policy planning.



7.2 AI Forecast Results Page

This page displays live carbon emission analysis for a selected country, showcasing the model's outputs and actionable insights. Key metrics such as current emissions, projected future emissions, model accuracy (R^2 score), and policy recommendations are highlighted at the top. The AI Forecast section visualizes historical and predicted CO_2 emissions trends on a clear, interactive graph, enabling users to quickly grasp how emissions could evolve and inform climate action planning.



7.3 Policy Recommendations Page

This page presents tailored climate policy recommendations generated by the platform's AI. Each recommendation card details a specific intervention such as implementing a carbon tax, setting renewable energy targets, or enhancing energy efficiency standards complete with expected impact, estimated implementation cost, and a projected timeline. The design helps decision-makers easily compare options and prioritize actions for reducing carbon emissions

Chapter 8

Conclusion

EcoVision AI – Global Carbon Insights & Forecasts is designed to assist policymakers, researchers, and environmental analysts in accurately predicting and understanding global carbon emission trends through advanced data-driven techniques. By integrating ARIMA and LSTM models within a unified ensemble framework, the system delivers reliable, interpretable, and scalable emission forecasts that adapt to complex, non-linear global datasets.

The platform automates every stage of the forecasting process — from data collection and preprocessing to model training, evaluation, and visualization — ensuring minimal manual intervention and maximum efficiency. It further extends its functionality by generating **AI-powered policy recommendations**, offering actionable guidance to governments and organizations striving to reduce emissions and meet sustainability goals.

Built using modern technologies such as **Flask**, **FastAPI**, **TensorFlow**, **React.js**, and **Docker**, EcoVision AI provides a responsive, scalable, and production-ready environment capable of handling large-scale datasets and delivering real-time analytical insights.

This project effectively demonstrates how Artificial Intelligence, Machine Learning, and modern web technologies can be harnessed to create intelligent forecasting systems that empower data-driven climate action and support global efforts toward a sustainable future

Chapter 9

Future Scope

While **Eco-Vision AI** successfully establishes a strong foundation for intelligent carbon emission forecasting and policy recommendation, there remains significant potential for future enhancements and research-driven expansion. The following directions outline how the system can evolve to achieve greater accuracy, scalability, and impact:

1. **Integration of Advanced Deep Learning Models**
   Incorporate transformer-based architectures, such as **Temporal Fusion Transformers (TFT)** or **Informer**, to further improve long-term forecasting accuracy and capture complex temporal dependencies across countries and sectors.

2. **Probabilistic and Causal Modeling**
   Introduce probabilistic forecasting frameworks and causal inference models to estimate not only emission outcomes but also the confidence levels and underlying drivers influencing those trends.

3. **Enhanced Data Diversity and Granularity**
   Expand data sources to include **satellite observations**, **real-time IoT sensor data**, and **sector-specific emission indicators** for industries like transportation, energy, and manufacturing to enable finer-grained analysis.

4. **MLOps and Continuous Model Deployment**
   Implement full **MLOps pipelines** for automated model retraining, version control, and deployment, ensuring that the system remains up-to-date with the latest data and algorithms.

5. **Multi-Language and Global Accessibility**
   Introduce multilingual support and region-specific dashboards to make EcoVision AI accessible to a broader global audience, promoting equitable climate insight sharing.

6. **Integration with Decision-Making Platforms**
   Collaborate with governmental and research institutions to integrate the system's forecasts into real-world policy dashboards, sustainability reports, and environmental monitoring frameworks.

References

[1] IEEE International Conference on Data Mining (ICDM), *"Time Series Forecasting of CO_2 Emissions Using ARIMA and LSTM Models,"* 2024.

[2] IEEE Transactions on Neural Networks and Learning Systems, *"Deep Learning Models for Predicting CO_2 Emissions in Power Systems,"* 2025.

[3] IEEE International Conference on Smart Grid Communications (SmartGridComm), *"Multi-Agent Systems for CO_2 Emission Reduction in Smart Grids,"* 2024.

[4] IEEE IGARSS Conference, *"AI-Based Global Carbon Mapping Using Remote Sensing and Machine Learning,"* 2025.

[5] World Bank Data Catalogue – *Global Energy and Emission Indicators Dataset,* https://data.worldbank.org.

[6] Our World in Data (OWID) – *CO_2 and Greenhouse Gas Emissions Dataset,* https://ourworldindata.org/co2-and-greenhouse-gas-emissions.

[7] European Commission – *EDGAR (Emission Database for Global Atmospheric Research),* https://edgar.jrc.ec.europa.eu/